Laporan Komprehensif Revisi & Desain Ulang PhysLang v2

Transformasi Menuju Domain-Specific Language (DSL) Fisika Komputasional

Anaf (Lead Architect) **Xinxinran (Co-Compiler Analyst)**

November 2026

Abstract

Dokumen ini menyajikan analisis menyeluruh, justifikasi teoritis, tata bahasa formal, serta rancangan teknis transisi **PhysLang** dari implementasi awal (*v1*) menuju **PhysLang v2**. Desain ulang ini dilakukan menyusul evaluasi akademik yang mengidentifikasi bahwa tata bahasa permukaan PhysLang v1 terlalu menyerupai bahasa pemrograman imperatif umum (*general-purpose*) dan belum menonjolkan abstraksi domain fisis secara kuat. PhysLang v2 mengadopsi paradigma *Computational Physicist POV* (gaya lembar kerja laboratorium) dengan menghilangkan *boilerplate* sintaksis yang tidak perlu, memperkenalkan pendefinisian rumus fisis sebagai warga kelas satu (*first-class formula declaration* `form`), proyeksi konversi satuan (`in`), serta percabangan kondisional (`if-else`) yang divalidasi ketat oleh matriks dimensi vektor $\mathbb{Z}^7$. Seluruh perubahan tata bahasa, aturan produksi (EBNF), struktur kuadrupel, tabel perbandingan, dan pemetaan perubahan modul kode sumber disajikan secara mendalam.

Contents

1 Latar Belakang dan Dekonstruksi Masukan Evaluator **2**
- 1.1 Kritik Akademik terhadap PhysLang v1 . . . 2
- 1.2 Analisis Akar Masalah: Disonansi Sintaksis Permukaan vs Semantik Internal . . . 2
- 1.3 Studi Komparasi Bahasa Domain: Dari R hingga Wolfram dan Frink . . . 2

2 Filosofi Desain PhysLang v2: Sudut Pandang Fisikawan **2**

3 Tata Bahasa Formal (EBNF Grammar) **3**
- 3.1 Aturan Leksikal (Tokens) . . . 3
- 3.2 Aturan Sintaksis (Syntactic Production Rules) . . . 3

4 Struktur Kuadrupel Formal: $G = (V_N, V_T, S, P)$ **4**
- 4.1 1. Himpunan Simbol Non-Terminal (V_N) . . . 4
- 4.2 2. Himpunan Simbol Terminal (V_T) . . . 4
- 4.3 3. Simbol Awal (S) . . . 4
- 4.4 4. Himpunan Aturan Produksi (P) . . . 5

5 Contoh Komparasi Kode Program: v1 vs v2 **5**
- 5.1 Implementasi PhysLang v1 (Imperatif Konvensional) . . . 5
- 5.2 Implementasi PhysLang v2 (Sintaks Laboratorium Fisika Murni) . . . 5

6 Daftar Fitur: Penambahan, Modifikasi, dan Penghapusan **6**

7 Pemetaan Perubahan pada Arsitektur Kompilator Kotlin **6**
7.1 1. Modul Leksikal (`lang.lexer`) 6
7.2 2. Modul AST (`lang.ast`) 6
7.3 3. Modul Parser (`lang.parser`) 6
7.4 4. Modul Penganalisis Semantik (`lang.semantic.SemanticAnalyzer`) 6
7.5 5. Modul Runtime Evaluator (`lang.ast.visitors.PhysicsEvaluator`) 7

8 Kesimpulan **7**

1 Latar Belakang dan Dekonstruksi Masukan Evaluator

1.1 Kritik Akademik terhadap PhysLang v1

Pada tahap pengujian awal tata bahasa, evaluator memberikan catatan kritis yang mencakup:

1. Sintaksis yang diajukan terkesan seperti bahasa pemrograman umum (*general-purpose language*) seperti JavaScript atau Kotlin mini.
2. Kurang menonjolnya abstraksi domain fisika pada tingkat aturan produksi tata bahasa permukaan (*surface grammar*).
3. Ketiadaan konstruksi fungsional atau formulasi eksplisit seperti pendefinisian entitas Area(a, b) yang lumrah ditemukan pada bahasa berbasis domain.
4. Ketiadaan struktur alur kendali (*control flow*) yang membuat bahasa terlihat belum memiliki kapabilitas pemrosesan bahasa terstruktur secara komprehensif.

1.2 Analisis Akar Masalah: Disonansi Sintaksis Permukaan vs Semantik Internal

Secara teknis, *backend* PhysLang v1 yang dibangun menggunakan Kotlin sebenarnya telah memiliki analisis semantik fisis yang sangat maju, di antaranya representasi aljabar linier vektor dimensi SI 7-elemen ($\mathbb{Z}^7$) serta validasi kompatibilitas dimensi pada pohon sintaks (AST).

Namun, seorang penilai tata bahasa kompilator mengevaluasi bahasa melalui **aturan sintaksis (EBNF)** yang tercantum pada dokumen laporan. Pada PhysLang v1, aturan produksi yang tampak adalah:

$$\text{VarDeclStmt} \rightarrow \texttt{"var"}\ \text{IDENTIFIER}\ (\texttt{"="}\ \text{Expression})^{?}$$
$$\text{PrintStmt} \rightarrow \texttt{"print"}\ \text{Expression}$$

Keberadaan kata kunci `"var"` dan `"print"` secara visual menyamarkan hakikat domain fisika menjadi sekadar program imperatif biasa yang kebetulan memiliki satuan pada literal angka.

1.3 Studi Komparasi Bahasa Domain: Dari R hingga Wolfram dan Frink

Dalam perancangan DSL, struktur bahasa harus mencerminkan cara berpikir praktisi domain tersebut:

- **Bahasa Statistik (R):** Memiliki operator formula khusus `~` (*Wilkinson-Rogers notation*) untuk merepresentasikan model regresi linear mpg $\sim$ wt + cyl. Karakteristik statistik dijadikan bagian dari tata bahasa, bukan sekadar pemanggilan fungsi pustaka.
- **Modelica:** Standar industri pemodelan fisika berbasis relasi kesetimbangan persamaan (*equation-oriented*), di mana variabel mengikat dimensi fisis tertentu (`Length`, `Force`) dan mengeliminasi konsep mutasi variabel imperatif.
- **Wolfram Language & Frink:** Memperlakukan konversi satuan dan operasi dimensi sebagai operasi tingkat pertama (*first-class operator*) menggunakan sintaks natural seperti `x in meter` atau operator proyeksi fisis.

PhysLang v2 mengambil posisi sebagai **Lightweight Interactive Physics DSL** yang mengedepankan kemudahan penulisan formula di laboratorium tanpa beban *solver* numerik yang rumit.

2 Filosofi Desain PhysLang v2: Sudut Pandang Fisikawan

PhysLang v2 membuang paradigma pemrogram perangkat lunak (*software engineer*) yang kaku dan menggantinya dengan paradigma fisikawan laboratorium:

1. **Deklarasi Variabel Nol-Boilerplate (`id = expr`):** Fisikawan di papan tulis tidak pernah menulis `var m = 10 kg`. Mereka menulis $m = 10$ kg. Oleh karena itu, kata kunci `var` dihapus total dan digantikan oleh penugasan langsung.

2. **Deklarasi Rumus Fisis Eksplisit (`form`):** Untuk mendefinisikan rumus seperti Area(a, b), fisikawan tidak memerlukan kurung kurawal berlapis atau kata kunci `return`. PhysLang v2 memperkenalkan konstruksi `form`:

   ```
   form Area(p, l) = p * l
   ```

   Penggunaan kata kunci `form` menghilangkan seluruh potensi ambiguitas pada parser antara pemanggilan fungsi (*CallExpr*) dan deklarasi rumus baru.

3. **Pelestarian Fitur Kustom Satuan (`unit`):** Fitur unggulan pembuatan satuan dinamis berbasis ekspresi fisis tetap dipertahankan dan disempurnakan:

   ```
   unit kph = 1000 m / 3600 s
   ```

4. **Operator Proyeksi Satuan (`in`):** Hasil evaluasi dapat langsung diamati dalam skala satuan tertentu secara deklaratif:

   ```
   inspect Ek in kJ
   ```

5. **Kondisi Batas Fisis (`if-else`):** Struktur alur kendali dihadirkan bukan untuk rekayasa algoritma umum, melainkan untuk mengevaluasi kondisi batas fisis (*physical boundary conditions*), titik kritis suhu, atau transisi fase, di mana operator pembanding $(<, >, ==)$ wajib berdimensi setara.

3 Tata Bahasa Formal (EBNF Grammar)

Berikut adalah spesifikasi tata bahasa bebas konteks (*Context-Free Grammar*) dalam notasi *Extended Backus-Naur Form* (EBNF) untuk PhysLang v2.

3.1 Aturan Leksikal (Tokens)

```
DIGIT             ::= [0-9]
INT_LITERAL       ::= DIGIT+
DOUBLE_LITERAL    ::= DIGIT+ "." DIGIT+
NUMBER_LITERAL    ::= INT_LITERAL | DOUBLE_LITERAL
STRING_LITERAL    ::= '"' [^"]* '"'
IDENTIFIER        ::= [a-zA-Z_][a-zA-Z0-9_]*

KEYWORD           ::= "unit" | "form" | "inspect" | "in" | "if" | "else" | "null"

ASSIGN_OP         ::= "=" | "+=" | "-=" | "*=" | "/="
COMPARE_OP        ::= "==" | "!=" | "<" | "<=" | ">" | ">="
ARITH_OP          ::= "+" | "-" | "*" | "/" | "^"
DELIMITER         ::= "(" | ")" | "{" | "}" | "," | "\n"
```

3.2 Aturan Sintaksis (Syntactic Production Rules)

```
Program           ::= ( Statement | "\n" )* EOF

Statement         ::= UnitDeclStmt
                    | FormulaDeclStmt
                    | IfStmt
                    | InspectStmt
                    | BlockStmt
                    | VarAssignStmt
                    | ExprStmt

UnitDeclStmt      ::= "unit" IDENTIFIER "=" Expression
```

```
FormulaDeclStmt  ::= "form" IDENTIFIER "(" ParameterList? ")" "=" Expression
ParameterList    ::= IDENTIFIER ( "," IDENTIFIER )*

IfStmt           ::= "if" "(" ConditionExpr ")" BlockStmt ( "else" BlockStmt )?
ConditionExpr    ::= Expression COMPARE_OP Expression

InspectStmt      ::= "inspect" Expression ( "in" IDENTIFIER )?
BlockStmt        ::= "{" ( Statement | "\n" )* "}"
VarAssignStmt    ::= IDENTIFIER ASSIGN_OP Expression
ExprStmt         ::= Expression

Expression       ::= ConversionExpr
ConversionExpr   ::= TermAddSub ( "in" IDENTIFIER )?
TermAddSub       ::= TermMulDiv ( ( "+" | "-" ) TermMulDiv )*
TermMulDiv       ::= TermPower ( ( "*" | "/" ) TermPower )*
TermPower        ::= UnaryExpr ( "^" UnaryExpr )*
UnaryExpr        ::= "-" UnaryExpr | PostfixExpr
PostfixExpr      ::= Primary ( "(" ArgumentList? ")" )*
ArgumentList     ::= Expression ( "," Expression )*

Primary          ::= QuantityLiteral
                   | NUMBER_LITERAL
                   | STRING_LITERAL
                   | IDENTIFIER
                   | "(" Expression ")"

QuantityLiteral  ::= NUMBER_LITERAL IDENTIFIER
```

4 Struktur Kuadrupel Formal: $G = (V_N, V_T, S, P)$

Berdasarkan definisi matematis Chomsky, tata bahasa PhysLang v2 didefinisikan oleh kuadrupel:

$$G = (V_N, V_T, S, P) \tag{1}$$

4.1 1. Himpunan Simbol Non-Terminal (V_N)

Entitas variabel tata bahasa yang digunakan dalam penurunan struktur sintaksis:

$$\begin{aligned} V_N = \{&\text{Program}, \text{Statement}, \text{UnitDeclStmt}, \text{FormulaDeclStmt}, \text{ParameterList}, \\ &\text{IfStmt}, \text{ConditionExpr}, \text{InspectStmt}, \text{BlockStmt}, \text{VarAssignStmt}, \\ &\text{ExprStmt}, \text{Expression}, \text{ConversionExpr}, \text{TermAddSub}, \text{TermMulDiv}, \\ &\text{TermPower}, \text{UnaryExpr}, \text{PostfixExpr}, \text{ArgumentList}, \text{Primary}, \\ &\text{QuantityLiteral}\} \end{aligned}$$

4.2 2. Himpunan Simbol Terminal (V_T)

Token leksikal fisik yang dikenali langsung oleh penganalisis leksikal:

$$\begin{aligned} V_T = \{&\text{NUMBER_LITERAL}, \text{STRING_LITERAL}, \text{IDENTIFIER}, \\ &\texttt{"unit"}, \texttt{"form"}, \texttt{"inspect"}, \texttt{"in"}, \texttt{"if"}, \texttt{"else"}, \texttt{"null"}, \\ &\texttt{"="}, \texttt{"+="}, \texttt{"-="}, \texttt{"*="}, \texttt{"/="}, \\ &\texttt{"=="}, \texttt{"!="}, \texttt{"<"}, \texttt{"<="}, \texttt{">"}, \texttt{">="}, \\ &\texttt{"+"}, \texttt{"-"}, \texttt{"*"}, \texttt{"/"}, \texttt{"\^{}"}, \\ &\texttt{"("}, \texttt{")"}, \texttt{"\{"}, \texttt{"\}"}, \texttt{","}, \texttt{"\textbackslash n"}\} \end{aligned}$$

4.3 3. Simbol Awal (S)

$$S = \text{Program}$$

4.4 4. Himpunan Aturan Produksi (P)

Himpunan relasi penurunan produksi yang memetakan elemen $V_N \rightarrow (V_N \cup V_T)^*$, persis sebagaimana didefinisikan pada subbab 3.2.

5 Contoh Komparasi Kode Program: v1 vs v2

Berikut adalah perbandingan konkret skrip yang mengevaluasi dinamika gerak dan gaya:

5.1 Implementasi PhysLang v1 (Imperatif Konvensional)

```
unit kph = 1000 m / 3600 s

var m = 1200 kg
var v = 90 kph
var a = 2.5 m/s^2

var F = m * a
var Ek = 0.5 * m * v^2

print "Gaya resultan: " + F
print "Energi kinetik: " + Ek
```

5.2 Implementasi PhysLang v2 (Sintaks Laboratorium Fisika Murni)

```
# 1. Deklarasi satuan pengukuran eksperimen
unit kph = 1000 m / 3600 s
unit Newton = 1 kg * m / s^2

# 2. Perumusan hukum fisika dasar (First-class Formula)
form Area(p, l) = p * l
form Gaya(m, a) = m * a
form EnergiKinetik(m, v) = 0.5 * m * v^2

# 3. Lembar data observasi (tanpa var)
massaUji = 1200 kg
kecepatan = 90 kph
percepatan = 2.5 m/s^2

# 4. Komputasi menggunakan formula
F = Gaya(massaUji, percepatan)
Ek = EnergiKinetik(massaUji, kecepatan)

# 5. Observasi dan proyeksi satuan
inspect F in Newton
inspect Ek in kJ

# 6. Pengujian kondisi batas sistem mekanik
batasKritis = 2500 Newton
if (F > batasKritis) {
    inspect "Peringatan: Terjadi over-stress mekanis!"
} else {
    inspect "Status beban normal."
}
```

Nama Fitur	Klasifikasi	Deskripsi Teknis & Rasionalisasi
`form`	**Fitur Baru**	Mendefinisikan fungsi fisis satu baris matematis murni. Menghilangkan ambiguitas pemanggilan fungsi dengan deklarasi formula.
`in`	**Fitur Baru**	Operator konversi satuan langsung. Memproyeksikan besaran $\mathbb{Z}^7$ ke unit target yang kompatibel dimensinya.
`if-else`	**Fitur Baru**	Kontrol alur untuk evaluasi kondisi batas fisis. Memeriksa relasi komparasi dengan verifikasi dimensi wajib setara.
`inspect`	**Modifikasi**	Menggantikan kata kunci `print` agar merefleksikan terminologi observasi sains di laboratorium.
`id = expr`	**Modifikasi**	Mengizinkan inisialisasi variabel langsung tanpa kata kunci pembuka deklarasi.
`var`	**Dihapus**	Dieliminasi total karena merupakan artefak bahasa pemrograman umum yang merusak estetika DSL sains.
`unit`	**Dipertahankan & Diperkuat**	Fitur inti perhitungan aljabar satuan kustom tetap dipertahankan 100% tanpa perubahan mekanisme komputasi.

Table 1: Matriks Perubahan Fitur Bahasa PhysLang v2

6 Daftar Fitur: Penambahan, Modifikasi, dan Penghapusan

7 Pemetaan Perubahan pada Arsitektur Kompilator Kotlin

Implementasi PhysLang v2 tetap mempertahankan arsitektur *Pratt Parser* dan *Tree-Walking Interpreter* yang ada, namun memperbarui beberapa modul kunci:

7.1 1. Modul Leksikal (`lang.lexer`)

- **Penambahan Token Type:** `Form`, `Inspect`, `In`, `If`, `Else`, `Less`, `Greater`, `LessEqual`, `GreaterEqual`, `EqualEqual`, `NotEqual`.
- **Penghapusan Token Type:** Menandai `Var` sebagai *deprecated* atau dihapus dari daftar kata kunci di `Keywords.kt`.

7.2 2. Modul AST (`lang.ast`)

- **Penambahan Node Statement:**
  - `FormulaDeclStmt(name: Token, params: List<Token>, body: Expr)`
  - `IfStmt(condition: Expr, thenBranch: Stmt, elseBranch: Stmt?)`
  - `InspectStmt(expr: Expr, targetUnit: Token?)`
- **Penyesuaian Statement Penugasan:** Menyatukan deklarasi dan penugasan ke dalam `AssignStmt` yang secara otomatis mendaftarkan simbol ke `Environment` jika belum ada.

7.3 3. Modul Parser (`lang.parser`)

- Mengimplementasikan metode `parseFormulaDecl()` untuk mengenali pola `form IDENTIFIER ( args ) = expr`.
- Mengimplementasikan metode `parseIfStmt()` yang mem-parse kondisi dalam kurung dan blok eksekusi cabang.
- Menambahkan operator infix `"in"` ke dalam tabel presedensi (*binding power*) ekspresi.

7.4 4. Modul Penganalisis Semantik (`lang.semantic.SemanticAnalyzer`)

- **Verifikasi Komparasi:** Memastikan bahwa ekspresi di sisi kiri dan kanan operator perbandingan $(<, >, ==)$ memiliki vektor eksponen dimensi $\mathbb{Z}^7$ yang identik. Membandingkan panjang

dengan massa akan langsung memicu `DiagnosticType.ERROR`.
- **Verifikasi Proyeksi `in`:** Memeriksa apakah satuan target pada operasi `in` memiliki dimensi yang kompatibel dengan ekspresi sumber.

7.5 5. Modul Runtime Evaluator (`lang.ast.visitors.PhysicsEvaluator`)

- Menyimpan objek `FormulaValue` di `Environment`. Ketika formula dipanggil via `CallExpr`, dibuat *local environment* untuk mengevaluasi parameter aktual dan mengembalikan `PhysicsValue`.
- Mengevaluasi cabang `IfStmt` berdasarkan nilai kebenaran ekspresi boolean.
- Menghitung faktor skala konversi satuan saat mengevaluasi pernyataan `InspectStmt` dengan klausa `in`.

8 Kesimpulan

Transformasi PhysLang menuju versi 2 berhasil merekonsiliasi ekspektasi akademik evaluator kompilator dengan fondasi komputasi yang telah dibangun. Dengan mengadopsi *Domain-Specific Grammar* yang dirancang khusus untuk fisikawan, PhysLang v2 memiliki identitas yang kuat, bebas dari kesan bahasa umum tiruan, namun tetap mempertahankan keunggulan perhitungan dimensi otomatis dan fleksibilitas pendefinisian satuan mandiri.